

Guía de integración Google Workspace para TI2

Taller de Integración II y Taller de Integración IV (TI4-16, Parte 1)

Segundo semestre 2026

1. Objetivo y alcance del Sprint 1

Esta guía prepara a TI2 para iniciar en Semana 2 la autenticación institucional con Google Workspace. Es una tarea de investigación y documentación (TI4-16). TI4 no implementa la integración en Backend, Web ni Database en este sprint.

La Parte 1 cubre solo lo necesario para comenzar:

- delimitar qué es autenticación institucional en este proyecto;
- explicar Google Workspace con OAuth 2.0 y OpenID Connect;
- explicar cómo encajan Better Auth y Convex;
- documentar los dominios @alu.uct.cl y @uct.cl;
- distinguir autenticación de autorización;
- identificar variables, callbacks y redirects sin valores reales;
- dejar referencias oficiales y un checklist corto.

Queda expresamente fuera de esta tarea y del Sprint 1 la integración con Google Calendar. Calendar se tratará más adelante como asistencia de TI4 a TI2, con guía técnica y apoyo de implementación cuando corresponda al alcance de agenda.

Esta guía complementa la guía de tecnologías, arquitectura y despliegue, y se coordina con TI2-3 (proveedor y sesión), TI2-14 (logout, errores y seguridad) y TI2-15 (pruebas y documentación).

2. Delimitación: qué sí es Sprint 1

Autenticación institucional en Sprint 1 significa: un usuario con cuenta Google de la universidad inicia sesión con OAuth, el backend reconoce su identidad y el cliente mantiene una sesión válida.

```
Cuenta Google UCT
  |
  v
OAuth 2.0 + OpenID Connect
  |
  v
Better Auth sobre Convex entrega identidad
  |
  v
Convex valida JWT en cada función
  |
  v
```

No es Sprint 1: roles finos por recurso, organizaciones, login con RUT o contraseña propia, doble factor, auditoría, ni Calendar. Esas piezas pertenecen a TI2-3, TI2-14 y TI2-15, o a sprints posteriores.

El stack de referencia ya está fijado: React con Vite y TanStack Router en Web, Convex como único backend y Better Auth como sistema de autenticación. No se usa TanStack Start porque Convex ya es el servidor; un segundo servidor Node solo agregaría complejidad innecesaria.

3. Google Workspace, OAuth 2.0 y OpenID Connect

Google Workspace es el proveedor de identidad. La universidad administra las cuentas, Google verifica la contraseña y el segundo factor, y nuestro sistema solo recibe una identidad verificada. Nunca vemos ni guardamos la contraseña institucional.

El flujo estándar es Authorization Code con OpenID Connect:

1. la Web redirige a `accounts.google.com` con cliente, scopes y `hd` cuando corresponda;
2. el usuario autentica con Google y aprueba los scopes;
3. Google devuelve un código a nuestro callback;
4. Better Auth intercambia el código por tokens y obtiene el perfil OpenID (`openid`, `email`, `profile`);
5. se crea la sesión y Convex emite su propio JWT para las funciones.

Los scopes del Sprint 1 son solo `openid`, `email` y `profile`. Pedir scopes de Calendar, Drive o correo queda prohibido en esta etapa.

En Google Cloud Console el proyecto necesita un OAuth Client de tipo Web application con:

- JavaScript origins autorizados: el origen de la Web y el Site URL de Convex;
- redirect URI autorizada: la ruta de callback de Better Auth en el Site URL de Convex, equivalente a `/api/auth/callback/google`;
- pantalla de consentimiento con logo, contacto y scopes mínimos;
- estado de publicación coherente con el uso (interno o externo según defina la universidad).

Esta guía no incluye valores reales de clientes, secretos ni URLs de deployment. Solo los nombres de las variables y la forma de las rutas.

4. Consola de Google Cloud paso a paso

Todo lo anterior se configura en `console.cloud.google.com`. Es la parte más propensa a errores porque un dato mal copiado bloquea el login con mensajes crípticos. Sigue este orden y verifica cada paso antes de pasar al siguiente.

4.1. 1. Proyecto

Crea o selecciona un único proyecto del equipo. Un solo proyecto es más simple de administrar y basta para este alcance. Todo lo que sigue vive dentro de ese proyecto: pantalla de consentimiento, credenciales y usuarios de prueba. No es necesario habilitar APIs adicionales en la biblioteca, porque el Sprint 1 solo usa autenticación (`openid`, `email` y `profile`) y no consume ninguna API de Google.

4.2. 2. Pantalla de consentimiento OAuth

Ruta: APIs y servicios, Pantalla de consentimiento. Define lo que ve el usuario cuando Google le pide permiso.

- Tipo de usuario **interno**: solo cuentas de la organización Workspace. No necesita usuarios de prueba y es la opción esperable si la universidad administra el Workspace.
- Tipo de usuario **externo**: cualquier cuenta Google. Mientras la app esté en modo de prueba, solo entran los usuarios registrados como test users; publicar exige verificación de Google.
- Datos obligatorios: nombre de la aplicación, correo de asistencia y dominios autorizados.
- Scopes: solo **openid**, **email** y **profile**. Son scopes no sensibles, por lo que no disparan el proceso de verificación extendida que exigen Calendar o Drive.

Si la universidad administra el Workspace, coordina con su TI qué tipo de usuario corresponde y si el administrador debe permitir la app: un admin puede bloquear apps de terceros y Google responde con `admin_policy_enforced`.

4.3. 3. ID de cliente OAuth

Ruta: APIs y servicios, Credenciales, Crear credenciales, ID de cliente OAuth. Tipo: **Aplicación web**.

- Orígenes de JavaScript autorizados: el origen de la Web en desarrollo (localhost con su puerto) y el origen de producción en Vercel.
- URIs de redirección autorizadas: la ruta de callback de Better Auth bajo el Site URL de Convex, con forma equivalente a `https://<deployment>.convex.site/api/auth/callback/google`. Debe coincidir exacta: esquema, mayúsculas y slash final. Cualquier diferencia devuelve `redirect_uri_mismatch`.
- Al crear el cliente, copia el Client ID y el Client Secret. El secreto se muestra una sola vez. Guárdalos fuera del repositorio y llévalos al deployment solo con `convex env set`.

4.4. 4. Errores típicos y su causa

redirect_uri_mismatch: el callback no está registrado o difiere en un carácter. Revisa la lista de URIs en Credenciales.

org_internal: cliente de tipo interno con una cuenta fuera de la organización. Entra con cuenta institucional o revisa el tipo de usuario.

admin_policy_enforced: el admin del Workspace bloquea apps de terceros. Lo resuelve el administrador, no el código.

invalid_client: secreto incorrecto o cliente eliminado. Regenera el secreto y actualiza el deployment.

Acceso bloqueado en modo de prueba: la cuenta no es test user o la app externa aún no se publica. Agrega la cuenta o avanza la publicación.

5. Dominios institucionales

El proyecto distingue dos dominios:

`@alu.uct.cl`: estudiantes.

`@uct.cl`: docentes, funcionarios y administración.

Better Auth soporta la opción `hd` del proveedor Google para exigir un hosted domain de Workspace. El parámetro `hd` sugiere el dominio en la pantalla de Google y, configurado en el proveedor, hace que Better Auth rechace tokens cuyo claim `hd` no coincida.

El patrón probado en cuestionarios-uct usa dos inicios lógicos sobre el mismo OAuth Client de Google, uno con `hd` de estudiantes y otro con `hd` de docentes, más una validación dura del sufijo del email en el mapeo de perfil para el rol docente. El parámetro `hd` por sí solo es una sugerencia de UX; la garantía real es validar el dominio en el servidor.

```
socialProviders: {
  google: {
    clientId: process.env.GOOGLE_CLIENT_ID,
    clientSecret: process.env.GOOGLE_CLIENT_SECRET,
    hd: "alu.uct.cl",
  },
}
```

Para dos poblaciones, TI2 puede optar por un proveedor con `hd` único si el sprint decide un solo dominio inicial, o por la variante de dos proveedores lógicos ya validada en cuestionarios-uct. Lo importante es que la decisión quede en el servidor (Better Auth sobre Convex), nunca solo en el frontend.

6. Cómo encajan Better Auth y Convex

Better Auth administra registro, login social, sesión y logout. Convex aporta ejecución, base de datos y verificación del JWT en cada función. La integración oficial ([@convex-dev/better-auth](https://github.com/convex-dev/better-auth)) hace que Better Auth corra como rutas HTTP dentro del deployment Convex y persista sus tablas en el componente `betterAuth`.

```
Web (Vite SPA)
|
+-- authClient.signIn.social({ provider: "google" })
|
v
https://<deployment>.convex.site/api/auth/*
(Better Auth sobre Convex)
|
+-- Google OAuth + callback
+-- tablas user, session, account, verification
|
v
Convex functions verifican ctx.auth.getUserIdentity()
```

Los archivos del backend son siempre los mismos:

```
convex/
|
+-- auth.config.ts proveedor JWT de Better Auth
+-- convex.config.ts registra el componente betterAuth
+-- http.ts expone /api/auth/* en Convex Site
+-- auth.ts createAuth: baseUrl, adapter,
| socialProviders.google, trustedOrigins
+-- schema.ts solo dominio; auth vive en el componente
```

Fragmento de forma (sin valores):

```

auth.config.ts:
  providers: [getAuthConfigProvider()]

convex.config.ts:
  app.use(betterAuth)

http.ts:
  authComponent.registerRoutes(http, createAuth)

auth.ts:
  baseUrl: process.env.BETTER_AUTH_URL
  database: authComponent.adapter(ctx)
  socialProviders: { google: { clientId, clientSecret } }
  plugins: [convex(), crossDomain()]

```

En la Web Vite (SPA, sin Start) el cliente es:

```

import { createAuthClient } from "better-auth/react";
import { convexClient, crossDomainClient } from
  "@convex-dev/better-auth/client/plugins";

export const authClient = createAuthClient({
  baseUrl: import.meta.env.VITE_CONVEX_SITE_URL,
  plugins: [convexClient(), crossDomainClient()],
});

```

Y el provider en la raíz:

```

const convex = new ConvexReactClient(
  import.meta.env.VITE_CONVEX_URL
);

<ConvexBetterAuthProvider client={convex} authClient={authClient}>
  <App />
</ConvexBetterAuthProvider>

```

El login social desde TanStack Router es una llamada al cliente, con `callbackURL` y `errorCallbackURL` del frontend:

```

await authClient.signIn.social({
  provider: "google",
  callbackURL: "/",
  errorCallbackURL: "/login",
});

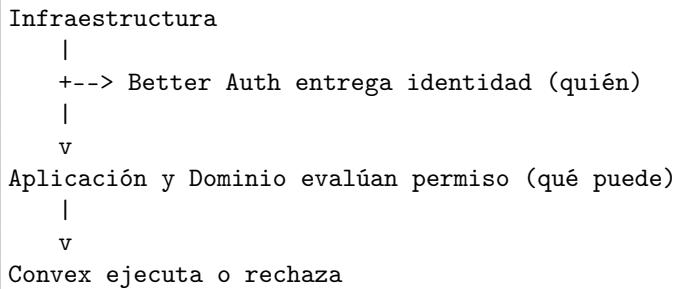
```

La sesión reactiva se lee con `authClient.useSession()`. La verdad autoritativa, sin embargo, siempre está en el backend con `ctx.auth.getUserIdentity()` o `authComponent.getAuthUser(ctx)`. El patrón verificado en ponderapp y reportevecino es: nunca aceptar un `userId` desde los argumentos para autorizar; la identidad viene del JWT.

7. Autenticación no es autorización

Autenticación: quién es el usuario. La resuelve Better Auth con Google y la entrega como identidad.

Autorización: qué puede hacer. La resuelven los casos de uso y el dominio, en el backend, a partir de esa identidad.



Ocultar un botón en React no es autorización. Toda regla (rol, recurso, pertenencia) debe verificarse en la función Convex antes de leer o mutar. Los roles finos y la matriz de permisos se definen en TI2-3 y se endurecen en TI2-14; en Sprint 1 basta con distinguir usuario autenticado de anónimo y dejar el punto de extensión para roles.

8. Variables, callbacks y redirects

Solo nombres. Ningún valor real pertenece a esta guía ni al repositorio.

Variable	Uso
GOOGLE_CLIENT_ID	OAuth Client de Google, solo servidor
GOOGLE_CLIENT_SECRET	Secreto OAuth, solo servidor
BETTER_AUTH_SECRET	Firma de sesiones, solo servidor
BETTER_AUTH_URL	Base URL de Better Auth en Convex
BETTER_AUTH_TRUSTED_ORIGINS	Orígenes extra permitidos
VITE_CONVEX_URL	Cliente Convex, pública
VITE_CONVEX_SITE_URL	Base de <code>/api/auth/*</code> , pública

Reglas:

- las secretas se configuran con `convex env set` en el deployment, nunca con prefijo `VITE_` ni en el bundle;
- las públicas `VITE_` son visibles para cualquiera; solo localizan servicios;
- `trustedOrigins` debe incluir el origen de desarrollo Vite (puerto de `dev`), el origen de producción en Vercel y el Site URL de Convex; se extiende por entorno con `BETTER_AUTH_TRUSTED_ORIGINS`;
- el callback Google es la ruta `/api/auth/callback/google` bajo el Convex Site URL; el frontend solo define `callbackURL` y `errorCallbackURL` de navegación post-login.

Archivos de ejemplo sin secretos: `apps/web/.env.example` con `VITE_CONVEX_URL` y `VITE_CONVEX_SITE_URL`; locales en `.env.local` fuera de Git.

9. Checklist para que TI2 comience en Semana 2

1. Completar la sección de Consola de Google Cloud: proyecto, pantalla de consentimiento (interno o externo), OAuth Client Web con redirect al Site URL de Convex y scopes `openid`, `email`, `profile`.
2. Fijar decisión de dominios: un `hd` inicial o los dos (`alu.uct.cl` y `uct.cl`) con validación de sufijo en el servidor.
3. Levantar `convex/auth.config.ts`, `convex.config.ts`, `http.ts` y `auth.ts` según la estructura oficial.

4. Configurar secretos solo con `convex env set` y publicar `.env.example` sin valores.
5. Conectar la SPA Vite con `authClient`, `convexClient`, `crossDomainClient` y `ConvexBetterAuthProvider`.
6. Implementar login y logout con `signIn.social` y `signOut`, más lectura con `useSession`.
7. Verificar en una query que `ctx.auth.getUserIdentity()` distinga autenticado de anónimo.
8. Registrar decisiones y dudas como comentarios en TI2-3 para coordinar con TI4.

10. Referencias oficiales

- Better Auth, Google social provider y opción `hd`: <https://better-auth.com/docs/authentication/google>
- Better Auth, uso de `signIn.social`: <https://better-auth.com/docs/basic-usage>
- Better Auth con Convex (guía marco React): <https://labs.convex.dev/better-auth/framework-guides/react>
- Better Auth con Convex (integración): <https://better-auth.com/docs/integrations/convex>
- Convex, cliente React: <https://docs.convex.dev/client/react/overview>
- Convex en hosting Vercel: <https://docs.convex.dev/production/hosting/vercel>
- Google Identity, OAuth 2.0 y OpenID Connect: <https://developers.google.com/identity/openid-connect/openid-connect>
- Google Cloud, configuración de pantalla de consentimiento y Client ID: <https://developers.google.com/identity/protocols/oauth2>
- Google Cloud, página de clientes OAuth (crear credenciales): <https://console.developers.google.com/auth/clients>
- Google Cloud, tipos de usuario y pantalla de consentimiento: <https://support.google.com/cloud/answer/10311615>
- TanStack Router con Vite: <https://tanstack.com/router/latest/docs/installation/with-vite>

Patrones internos verificados: cuestionarios-uct (`apps/web/src/server/auth/index.ts`, `src/lib/auth/client`, `src/lib/constants.ts`) para Google con `hd` y validación por dominio; ponderapp (`apps/backend/convex/auth`, `apps/web/src/lib/auth-client.ts`) y reportevecino (`apps/backend/convex/auth.ts`, `apps/backend/convex`, `apps/dashboard/src/integrations/convex/provider.tsx`) para Better Auth sobre Convex y verificación de sesión en backend.